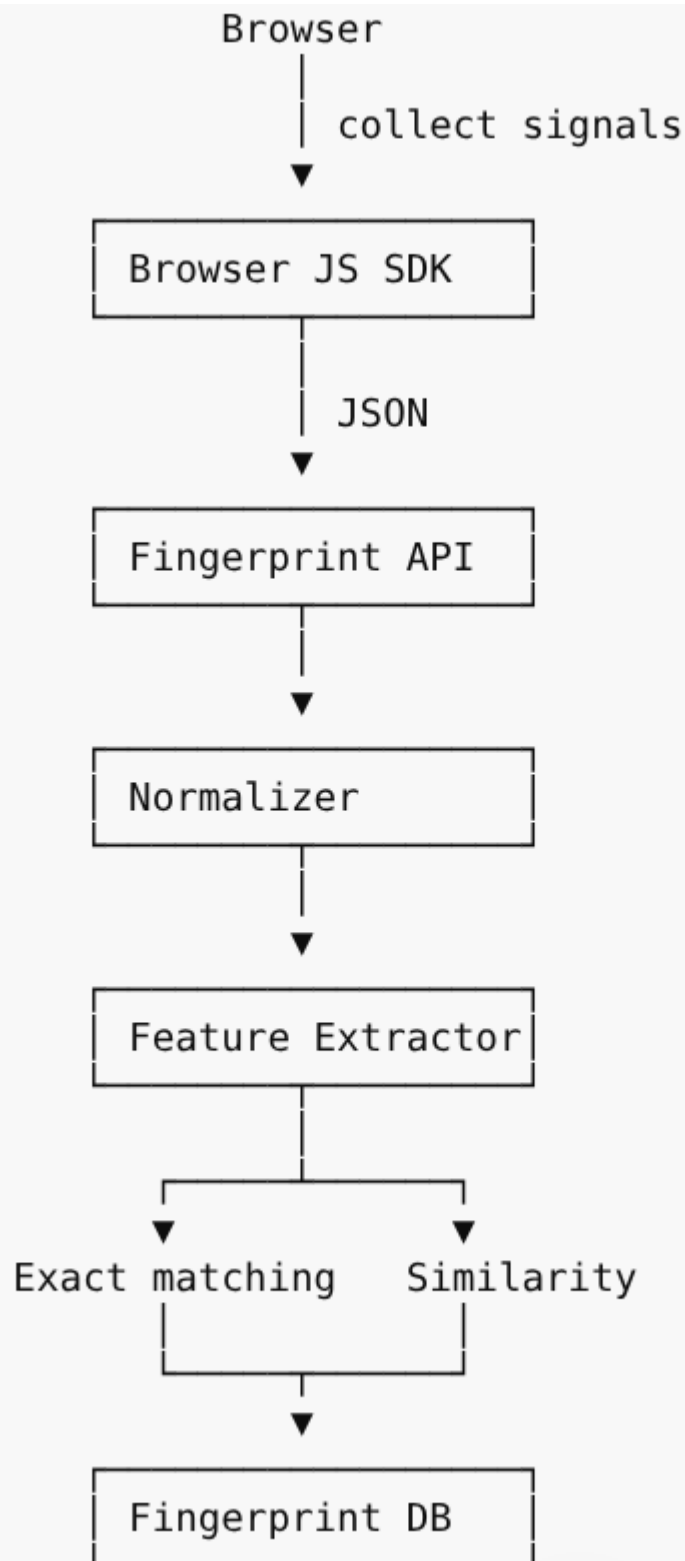




POST /v1/fingerprint

request:

```
{
  "userAgent": "...",
  "platform": "Win32",
  "language": "en-IN",
  "timezone": "Asia/Kolkata",
  "screen": {
    "width": 1920,
    "height": 1080,
    "pixelRatio": 1
  },
  "hardware": {
    "cpuCores": 8,
    "memory": 16
  }
}
```

response:

```
{
  "fingerprintId": "fp_8f3a91c2",
  "confidence": 0.94,
  "isNew": false
}
```

for a POC, this is already enough to demonstrate the basic concept.

6. I would actually create TWO IDs

This is an interesting design choice.

Raw fingerprint

Represents the collected characteristics:

FP_A



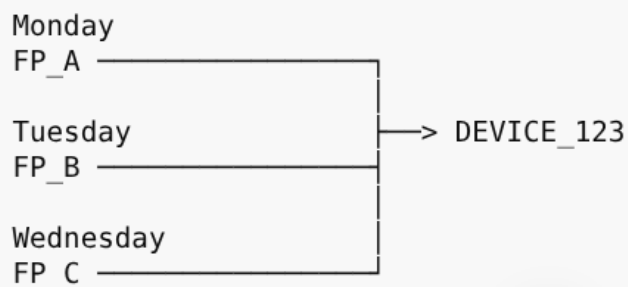
Device identity

Represents your conclusion after comparing fingerprints:

DEVICE_123



For example:



Even if FP_A , FP_B , and FP_C aren't exactly identical, your matching system



7. Exact matching vs similarity matching

Your POC should demonstrate both.

Method 1 — Exact

```
fingerprint_A == fingerprint_B
```

Very simple.

But fragile.

Method 2 — Weighted similarity

Suppose you have:

Signal	Weight
WebGL	25%
OS/platform	15%
Browser	15%
Timezone	10%
Screen	10%
Language	5%
CPU cores	5%

You compare two fingerprints.

For example:

```
WebGL      → same      +25
OS          → same      +15
Browser     → same      +15
Timezone    → same      +10
Screen      → different  +0
Language    → same      +5
CPU         → same      +5
           ---
           75%
```

Then:

```
similarity = 75%
```

Your application decides how to interpret that score.

For a POC, I'd avoid calling the score "truth." Call it something like:

```
similarityScore
```

because fingerprinting is inherently probabilistic.



8. What should your microservice do?

I'd keep the first version small.

API 1 — Create/identify

```
POST /v1/fingerprint
```



Input:

```
{  
  "signals": {...}  
}
```



Output:

```
{  
  "fingerprintId": "...",  
  "deviceId": "...",  
  "isNew": false,  
  "similarityScore": 0.92  
}
```



API 2 — Compare

POST /v1/fingerprint/compare

Input:

```
{
  "fingerprintA": {...},
  "fingerprintB": {...}
}
```

Output:

```
{
  "similarityScore": 0.91,
  "matched": true
}
```

API 3 — Get history

GET /v1/device/{deviceId}

Could return:



API 3 — Get history

GET /v1/device/{deviceId}

Could return:

```
{
  "deviceId": "device_123",
  "firstSeen": "...",
  "lastSeen": "...",
  "observations": 42
}
```

9. Suggested POC technology stack

You don't need anything complicated.

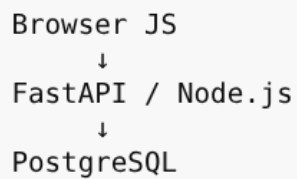
For example:



```
graph TD; Frontend --> JS[JavaScript SDK]; JS --> API[Node.js / Java / Go / Python API]; API --> Redis; Redis --> PostgreSQL;
```

Frontend
↓
JavaScript SDK
↓
Node.js / Java / Go / Python API
↓
Redis
↓
PostgreSQL

I'd personally keep the first POC as:



```
graph TD; Browser[Browser JS] --> API[FastAPI / Node.js]; API --> PostgreSQL;
```

Browser JS
↓
FastAPI / Node.js
↓
PostgreSQL

Add Redis later if you actually need caching/high throughput.

10. Important: fingerprint ≠ authentication

This is another important point.

Don't build your system around:

```
fingerprint == proof of identity
```

Instead:

```
fingerprint == signal
```

For example:

```
Login request
├── Password/passkey → authentication
├── Device fingerprint → device signal
├── IP/network → network signal
└── Behaviour → additional signal
```

Your application can combine these signals for things like:

- detecting a new device

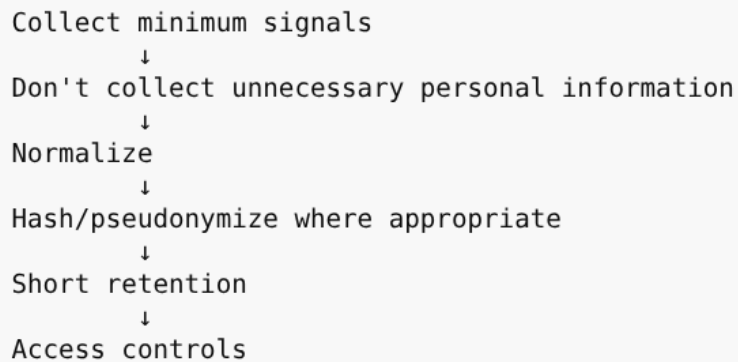
Your application can combine these signals for things like:

- detecting a new device
- detecting unusual login environments
- fraud/risk analysis
- account security
- session analysis
- bot/automation detection

11. Privacy is important

Device fingerprinting can be privacy-sensitive because combining enough signals can make a browser/device relatively identifiable.

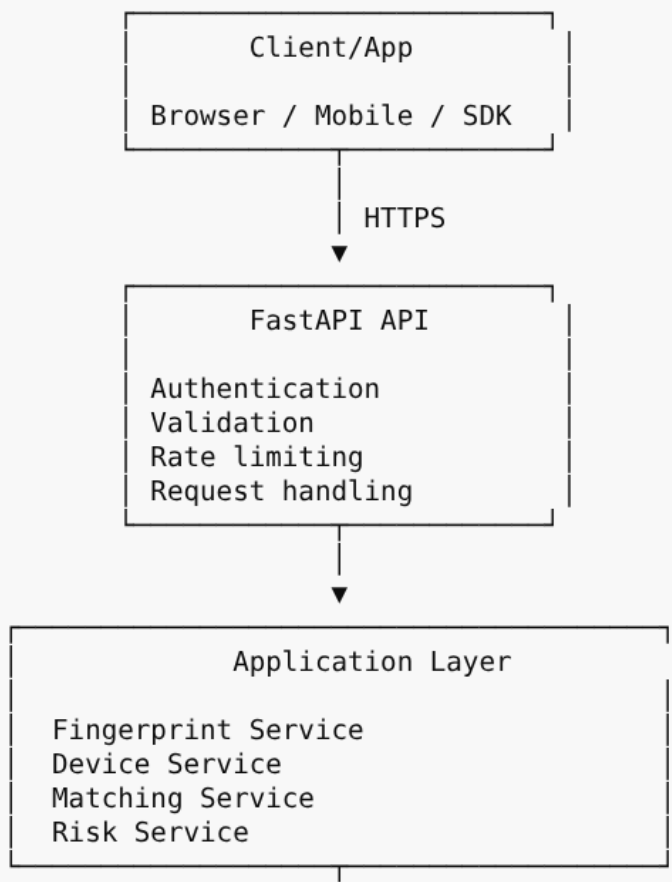
For your POC, I'd therefore design it with:

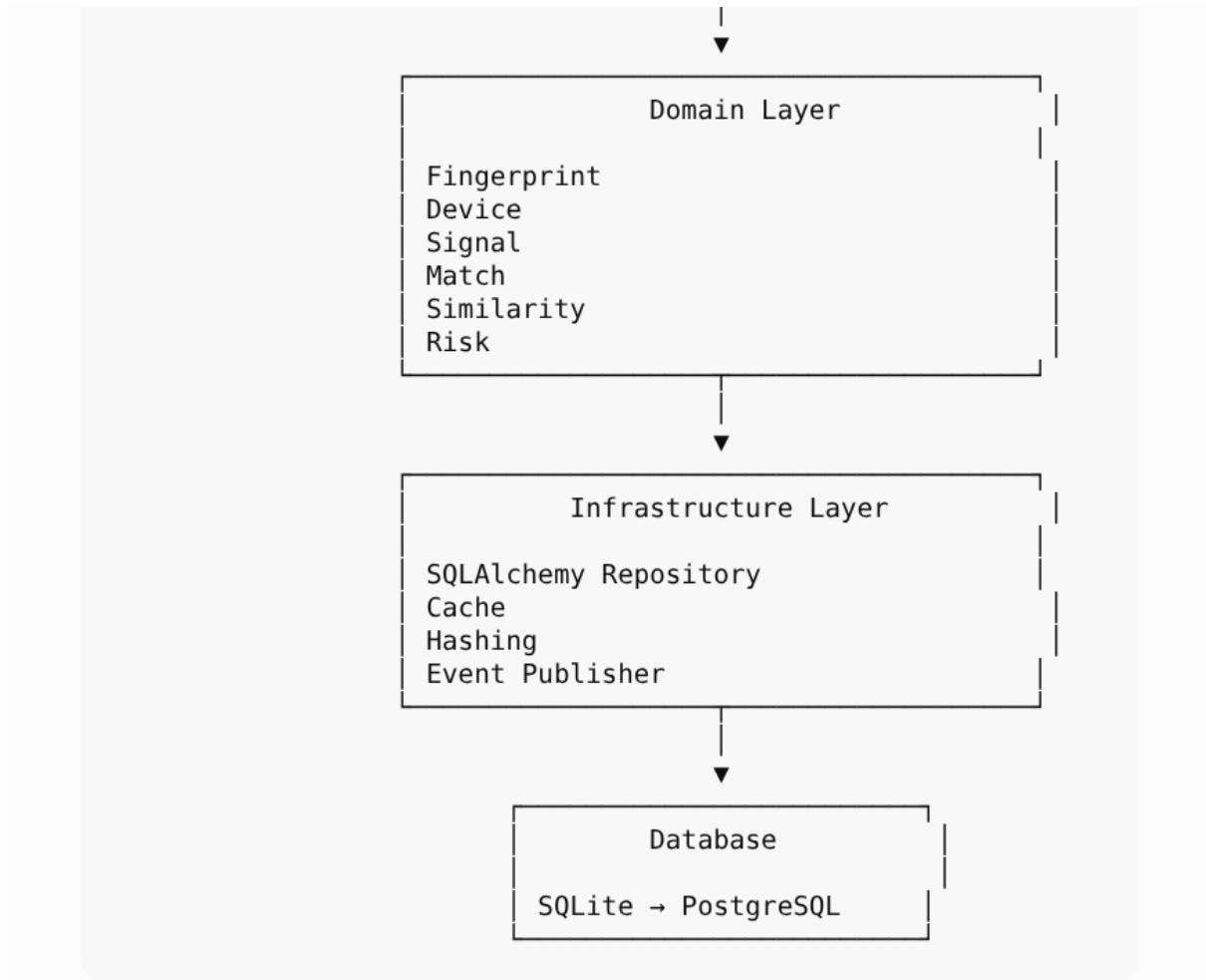


```
graph TD; A[Collect minimum signals] --> B[Don't collect unnecessary personal information]; B --> C[Normalize]; C --> D[Hash/pseudonymize where appropriate]; D --> E[Short retention]; E --> F[Access controls];
```

Also consider the applicable privacy laws and browser policies for wherever the service will actually be deployed.

1. Overall architecture





device-fingerprint-service/

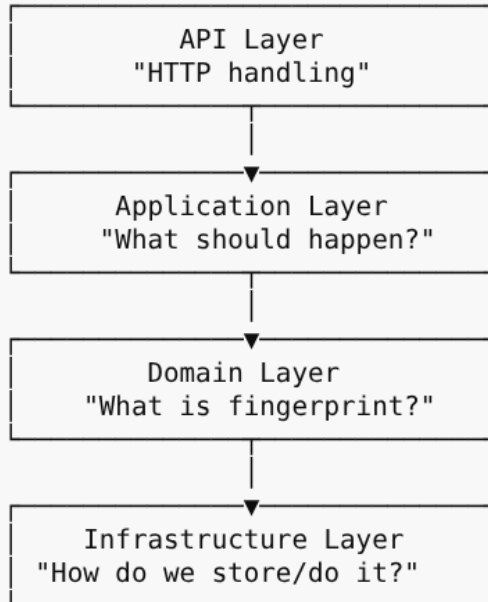
```
├── app/
│   ├── main.py
│   ├── config/
│   │   ├── settings.py
│   │   └── logging.py
│   ├── api/
│   │   ├── dependencies.py
│   │   └── v1/
│   │       ├── router.py
│   │       ├── fingerprint.py
│   │       ├── device.py
│   │       └── health.py
│   └── application/
│       ├── fingerprint_service.py
│       ├── device_service.py
│       ├── matching_service.py
│       └── risk_service.py
```

```
├── domain/
│   ├── entities/
│   │   ├── fingerprint.py
│   │   ├── device.py
│   │   └── observation.py
│   ├── value_objects/
│   │   ├── fingerprint_id.py
│   │   ├── device_id.py
│   │   └── similarity_score.py
│   ├── services/
│   │   ├── fingerprint_generator.py
│   │   └── fingerprint_matcher.py
│   └── repositories/
│       ├── fingerprint_repository.py
│       └── device_repository.py
├── infrastructure/
│   ├── database/
│   │   ├── session.py
│   │   ├── models/
│   │   │   ├── fingerprint.py
│   │   │   ├── device.py
│   │   │   └── observation.py
│   │   └── repositories/
│   │       ├── fingerprint_repository.py
│   │       └── device_repository.py
│   └── hashing/
│       └── sha256.py
```

```
├── cache/
│   └── redis.py
├── schemas/
│   ├── fingerprint.py
│   ├── device.py
│   └── common.py
├── tests/
│   ├── unit/
│   ├── integration/
│   └── api/
├── alembic/
│   └── versions/
├── alembic.ini
├── pyproject.toml
├── Dockerfile
├── docker-compose.yml
├── .env.example
└── README.md
```

3. The four important layers

Think about the architecture like this:



This is essentially a **Clean Architecture / Hexagonal Architecture style**.

It maps very nicely to Spring Boot later.

4. API layer

FastAPI should be thin.

For example:

```
POST /api/v1/fingerprints
```

The endpoint should roughly do:

```
@router.post("/fingerprints")
def create_fingerprint(
    request: FingerprintRequest,
    service: FingerprintService = Depends(...)
):
    return service.process(request)
```

Don't put this inside the controller:

```
# DON'T

@router.post("/fingerprints")
def fingerprint(request):

    normalize_data()

    calculate_hash()

    compare_database()

    calculate_similarity()

    create_device()

    update_device()

    save_database()
```

That becomes difficult to maintain.

Instead:

```
Controller
  ↓
Application Service
  ↓
Domain
  ↓
Repository
  ↓
Database
```

5. Application layer

This is where your **use cases** live.

For example:

```
FingerprintService
    process_fingerprint()

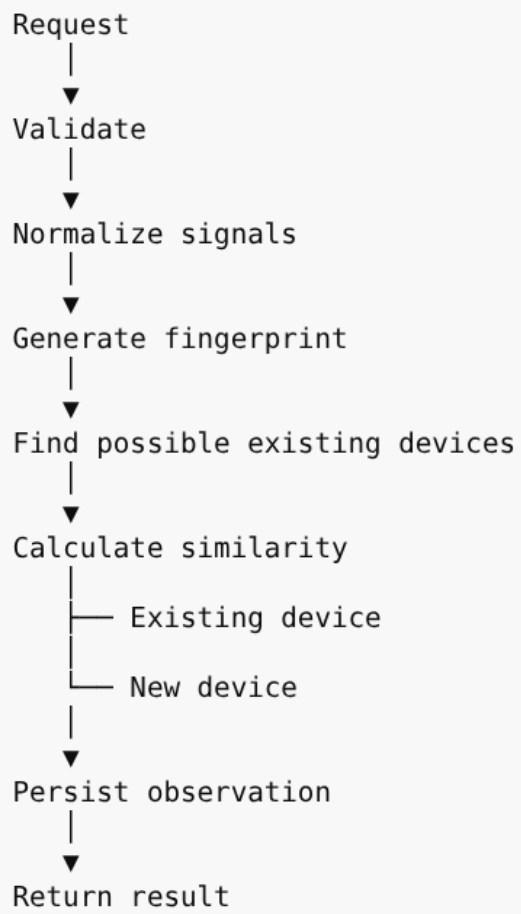
DeviceService
    get_device()
    get_device_history()

MatchingService
    compare_fingerprints()

RiskService
    calculate_risk()
```



Conceptually:



6. Domain layer

This is the **heart of your system**.

Your domain should not care whether you're using:

```
SQLite
PostgreSQL
MySQL
```



or:

```
FastAPI
Spring Boot
```



It should understand concepts such as:

```
Device
Fingerprint
FingerprintSignal
Observation
SimilarityScore
```



For example:

11. Why this matters for Spring Boot

Your eventual mapping becomes almost one-to-one:

Python	Spring Boot	
FastAPI Router	@RestController	
Pydantic	DTO / Jakarta Validation	
Application Service	@Service	
Domain Entity	Domain Entity	
Repository Protocol	Repository Interface	
SQLAlchemy	JPA/Hibernate	
Alembic	Flyway/Liquibase	
Dependency Injection	Spring DI	
pytest	JUnit	
SQLite	PostgreSQL	
Redis	Redis	

Milestone 1 — Basic fingerprint

```
Browser
  ↓
POST /fingerprints
  ↓
Normalize
  ↓
SHA-256
  ↓
SQLite
```



Milestone 2 — Device association

```
Fingerprint
  ↓
Existing fingerprint?
  ↓
Yes → existing device
No  → create device
```



Milestone 3 — Similarity

```
Fingerprint A
  +
Fingerprint B
  ↓
Similarity engine
  ↓
Score
```



Milestone 4 — Production concerns

```
Authentication
Rate limiting
Logging
Metrics
Error handling
Security
Privacy
```



Milestone 5 — Scale

SQLite
↓
PostgreSQL

Local cache
↓
Redis

Single instance
↓
Multiple instances

